Shared Memory Parallelism Report

November 2020

Contents

1 README **1**
1.1 parallel.c 1
1.2 sequential.c 1
1.3 Output 1

2 Implementation **2**
2.1 Array Generation and Input 2
2.2 Computation 2
2.3 Avoiding Race Conditions 2
2.4 Precision 3

3 Correctness Testing **4**

4 Scalability Investigation **6**
4.1 General 6
4.2 Efficiency 18
4.3 Speedup 20

1 README

1.1 parallel.c

The compiled C file (compile with the command `gcc parallel.c -lpthread -lrt`) can be run with or without the following arguments, in the stated order:

1. Dimension of input array. For example, `5` for a 5x5 array. Must be > 2 to have mutable elements Default is `18`.

2. Precision. For example, `0.1`. Default is `0.01`.

3. Number of threads to run. Any number of threads up to and including the number of mutable array elements will run, but note the maximum number of CPU cores on one Balena node (16). Default is `16`.

4. Custom array; input the name of the text file (`.txt`) containing the array as a command line argument to import the array. Array must match input dimension. Separate each value with one space only. Default is an array of random integers between 0 and the array dimension.

Commands *must* be entered in this order, and with no gaps. For example, `./parallel.out 20 0.1` is permissible for a `20x20` array and a precision of `0.1`, but `./parallel.out 20 10` will not run a `20x20` array with `10` threads.

1.2 sequential.c

The compiled C file (compile with the command `gcc sequential.c -lrt`) can be run using the same arguments as above and with the same caveats, with the exception that 'Number of Threads' may not be specified (as the program runs serially using a single thread of execution), and hence argument 4 (filename) should be entered as argument 3 if it is to be used. Identical defaults are used.

1.3 Output

By default, the provided files will output their results in the form of a labelled file: either `parallel-results-$dimension-$precision-$threadCount.txt` for `parallel.c`, or `sequential-results-$dimension-$precision.txt` for `sequential.c`. Here `$dimension` refers to the array dimension, `$precision` to the input precision, and `$threadCount` to the number of threads being used, either as defined via the command line input outlined above, or set to their default values of `18`, `0.01`, and `16` respectively.

2 Implementation

2.1 Array Generation and Input

The default array generated will be a random array of integers between 0 and the array dimension (either the input array dimension or the default array dimension of 18. To use custom arrays, see README.

The default array is generated by producing a random integer between 0 and `RAND_MAX` (a system-dependent value guaranteed to be at least 32767) and computing this value modulo the array dimension. Provided a large enough difference between `RAND_MAX` and the array dimension, this produces a pseudo-random integer between 0 and the array dimension. To ensure both the parallel and the sequential programs generate the same array, the random seed in both programs is set to the same value (in the case of the scalability investigation herein, it is set to 1), and can be altered in the code for experimental purposes.

2.2 Computation

This implementation of the relaxation technique uses `pthreads` to produce a solution, using a `do-while` loop that continues until no elements in the array change by more than the defined precision in the most recent. In each iteration, all threads operate on at least $(arraySize - 2)^2 \ /threadCount$ mutable array elements, with the 'remaining' elements (if the number of mutable elements, $arraySize - 2$, is not divisible by the number of threads) operated on by one thread each. For example, where a 20x20 array was being operated on by 16 threads, there would be 324 (18×18) mutable elements, so 12 threads would operate on 20 (324/16) array elements each and 4 (324 % 16) threads would operate on 21 each. This ensures that work is as evenly divided amongst the threads as possible, maximising the gains from parallelising operations and minimising the time threads (and hence processors) spend idle.

2.3 Avoiding Race Conditions

In order to avoid a race condition whereby threads operating on values from different iterations, two different checks were required. Firstly, two separate arrays are used to store values: one to hold the values produced from the last iteration, and one to hold those produced by threads in the current iteration. Threads then read values from the former array when calculating the

mean of an element, before writing to latter; at the end of the iteration all threads (including main) call `pthread_barrier_wait()` on the same barrier, a barrier that was initialised before the threads were created. This ensures that all threads have finished iterating their values, and the iterable array is 'finalised' for this iteration.

Each thread other than the main control thread (that runs `main()`) then calls `pthread_barrier_wait()` again - this allows for the control thread to run flag updates and to check if the program is finished before the other threads resume, while the use of barriers allows for the re-use of threads, reducing the overhead of thread creation and termination. Additionally, the two array pointers are swapped using a temporary 2D array pointer as an intermediary, providing a new 'empty' array with the same boundary values to iterate on, and an up-to-date array to read values to average from. This ensures that threads cannot 'run ahead' by multiple iterations, or operate on an array where some values have been updated and others have not. This process could have further been improved by the implementation of a 'thread pool', but unfortunately this was not completed.

2.4 Precision

Iteration continues until all elements in the array, when iterated again, change by less than the given precision. This process is also parallelised. A Boolean flag, set to `true` at the beginning of each iteration. Each time a thread operates on an element a precision check is then conducted: if the absolute value of the difference between the former value and the new value of the element is greater than the precision, the flag is changed to false, and remains false for the rest of the iteration - else it remains true. The program then continues iterating until the flag is set to true even once all threads have called `pthread_barrier_wait()`, at which points the program outputs the result and exits.

3 Correctness Testing

In order to conduct correctness testing the results produced by the parallel program must be compared to a 'ground truth', in this case produced by a sequential program utilising the same algorithm. It is assumed that this sequential program produced 'correct' outputs: these outputs were manually verified for several small arrays and large precisions, and for several larger arrays was also verified against the output arrays of several other students.

In order to automate correctness testing for a large number of arrays and a variety of precisions, a bash script was produced that could schedule multiple jobs at once via a loop that iterated through various array sizes, precisions, and additionally several different thread quantities for the parallel version (in an attempt to expose any rare race conditions). Both the parallel and sequential code was then altered slightly to write the output array to a file, uniquely named for that particular run - for example, `resultSequential-1000-0.01` would identify the output of the sequential program run on an array size of 1000 and to a precision of 0.01. The content of the arrays itself was manually altered between runs of the script - for example, one set of arrays consisted of randomly allocated 1s and 0s, whilst another consisted of a set boundary value whilst the interior of the array was populated with 0.0s.

For example, the output from the running of the sequential program on a 7x7 array generated using the default array generator and to a precision of 0.01 is:

```
1.000000,4.000000,2.000000,5.000000,1.000000,3.000000,3.000000,
2.000000,3.143514,2.965080,3.510914,2.774421,3.299451,4.000000,
6.000000,3.618169,3.193568,3.322523,3.275012,3.432559,3.000000,
0.000000,2.123264,2.896069,3.285966,3.598085,4.143471,6.000000,
1.000000,1.997210,2.956822,3.363938,3.663266,3.561600,3.000000,
2.000000,1.896416,3.597656,3.525052,4.156997,3.427353,4.000000,
5.000000,0.000000,6.000000,3.000000,6.000000,2.000000,2.000000,
```

This output will then be written to the file `resultSequential-7-0.01`, and compared against the results from `resultParallel-$numThreads-7-0.01` for `numThreads` 2 through 32. Running a simple script, such as the below, will allow the generation of multiple output files for various array sizes and precisions, with manual alterations in the content of the arrays:

```
for arraySize in 50 100 150 200 250 300 350 400
do
```

```
for threadNum in 2 4 6 8 10 12 13 14 15 16 17 18 19 20 22 24 26 28 30 32
do
for priority in 1.0 0.1 0.01 0.001 0.00001 0.0000001
do

echo "#!/bin/bash" >> singleScaleTest-$arraySize-$priority-$threadNum.slurm
echo "#SBATCH --account=cm30225" >> singleScaleTest-$arraySize-$priority-$threadNum.slurm
echo "#SBATCH --partition=teaching" >> singleScaleTest-$arraySize-$priority-$threadNum.slurm
echo "#SBATCH --job-name=Parallel-$arraySize-$priority-$threadNum" >> singleScaleTest-$arraySize-$priority-$threadNum.slurm
echo "#SBATCH --time=00:15:00" >> singleScaleTest-$arraySize-$priority-$threadNum.slurm
echo "#SBATCH --output=parallel-results-$arraySize-$priority-$threadNum.txt" >> singleScaleTest-$arraySize-$priority-$threadNum.slurm
echo "#SBATCH --nodes=1" >> singleScaleTest-$arraySize-$priority-$threadNum.slurm
echo "perf stat ./parallel.out $arraySize $priority $threadNum" >> singleScaleTest-$arraySize-$priority-$threadNum.slurm
sbatch singleScaleTest-$arraySize-$priority-$threadNum.slurm

echo "#!/bin/bash" >> singleScaleTest-$arraySize-$priority.slurm
echo "#SBATCH --account=cm30225" >> singleScaleTest-$arraySize-$priority.slurm
echo "#SBATCH --partition=teaching" >> singleScaleTest-$arraySize-$priority.slurm
echo "#SBATCH --job-name=Sequential-$arraySize-$priority" >> singleScaleTest-$arraySize-$priority.slurm
echo "#SBATCH --time=00:15:00" >> singleScaleTest-$arraySize-$priority.slurm
echo "#SBATCH --output=sequential-results-$arraySize-$priority.txt" >> singleScaleTest-$arraySize-$priority.slurm
echo "#SBATCH --nodes=1" >> singleScaleTest-$arraySize-$priority.slurm
echo "perf stat ./sequential.out $arraySize $priority" >> singleScaleTest-$arraySize-$priority.slurm
sbatch singleScaleTest-$arraySize-$priority.slurm

done
done
done
```

The following script can then be used to compare the contents of the output files to ensure they are identical:

```
if cmp -s "resultParallel-$numThreads-$arraySize-$precision.txt" "resultSequential-$arraySize-$precision.txt"; then

echo "$arraySize: Correct"

else

echo "$arraySize: Incorrect"

fi
```

The bash `cmp` command is used to compare the contents of the files containing the outputs of the sequential program with the parallel program for a number of different thread counts. The array types measured included random 1s and 0s, random boundary integers and internal values of 0, and random floats in various ranges. These array types can all be tested with the custom array input option, and use of the above scripts.

On all tests run, on arrays from size 3x3 to 10,000x10,000, with multiple precisions and array types, the contents of the files were identical. Hence it can be concluded that the parallel implementation provided is 'correct'.

4 Scalability Investigation

4.1 General

In order to assess how the parallel implementation scaled, it was necessary to develop a script that could automate running large numbers of jobs. These jobs would be performed using various array dimensions, precisions, and numbers of threads, and collect statistics including running time, CPU utilisation, and number of iterations to complete. This script ran 'standard' versions of the included code (no compile-time optimisations), and is included in the Correctness Testing section.

The primary array type ('type' referring to the content of the array) used for these tests was a random array of values between 0 and the array dimension, as was mentioned above as the current default setting for the program, and in particular an examination of this array type to a precision of 0.01. This was due to a relatively strong correlation between array dimension and the number of iterations it took to produce a solution; a random seed of 1 was selected for this reason.

A variety of array types were experimented with, but for many of the metrics detailed in this report produced similar results, if at a different scale. For example, initial tests were conducted with an array with a boundary of 1.0, and otherwise filled with 0.0s. This array could scale to just over `7000x7000` on a precision of 0.01 before hitting the Balena time cap of 15 minutes, though this array also scaled in manner that was not generalisable, as iterating the array simple resulted in values that got smaller as they moved towards the centre, and so after a certain size a larger array simply resulted in a larger 'middle' section of 0.0s. In particular, the use of an array of random 1.0s and 0.0s displayed some interesting properties. Where array types other than the 'default' have been used it has been noted; in particular the above described array is listed as the '1s and 0s' array type.

The first (and simplest) metric to measure was simple program running time for arrays of different dimensions. Across all thread quantities measured (see Figure 1) running time (measured using the `perf` tool) increasing broadly exponentially as array size increased. This held as long as increased array size increased with the number of iterations it took to process. This is notable number of iterations increased linearly with array size (see Figure 2), though this occurred in two notable segments - between array dimensions of 0 and 250 and 250 upwards. This was linear scaling an effect of the chosen random

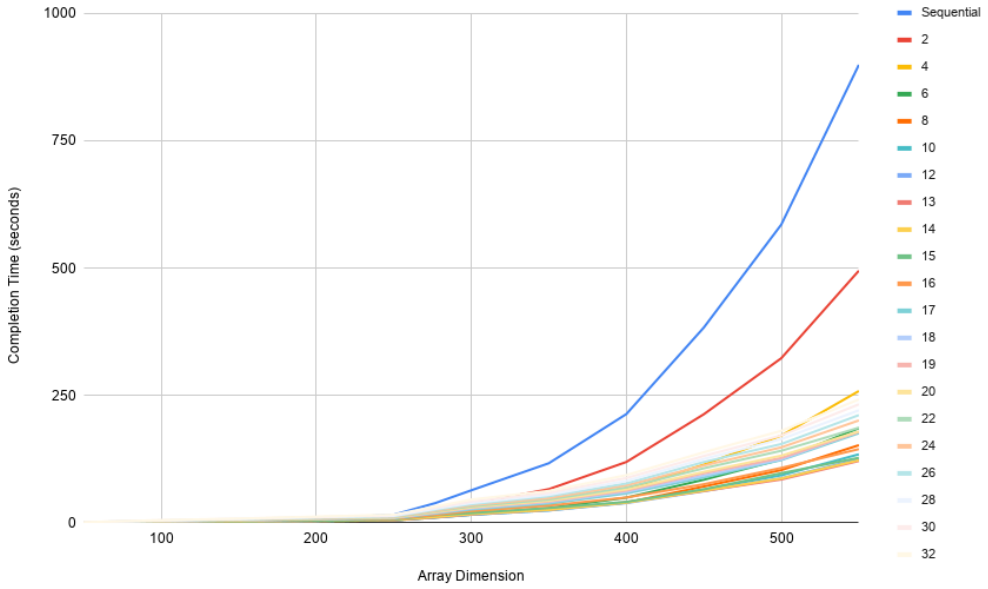

Figure 1: Program total running time vs array dimension for 1-32 threads

seed, as it was not demonstrated when using alternate seeds. The rate of linear increase had a direct effect on the rate of exponential increase - as can be seen in Figure 3, an increase in array size from 0x0 to 250x250 displays a similar exponential increase, but at a far slower rate than is demonstrated from 250x250 to 550x550, when rate of linear increase in iteration number is far greater

The compounded effects of an increase in the number of elements each thread has to process and the number of iterations taken to process the array, the former increasing quadratically with array dimension and the latter linearly, likely resulted in this scaling. Additionally, the fact that the range of values in the array increases with array size also likely had a significant effect, as this exponential behaviours is not seen in other array types, such as when arrays are formed from a random series of 1s and 0s (see Figure 4).

Optimal thread count was largely dependent on dimension. A trend displayed across all arrays types tested was that as array size increased the optimal thread count trended upwards (though at varying rates depending on the array type) towards a limit of at least 15 on conducted tests. As can be seen in both Figure 1 and more clearly in Figure 6, for the smaller

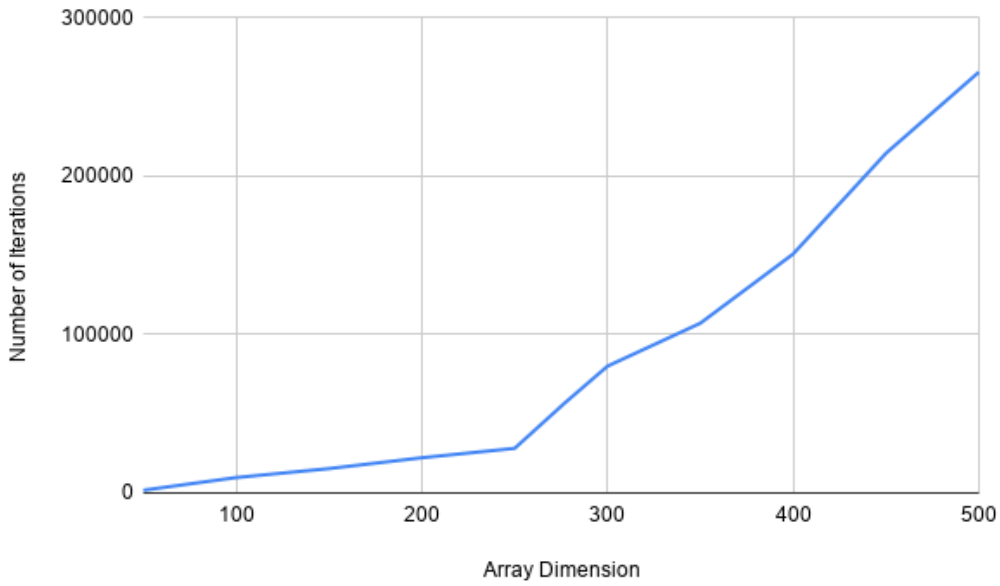

Figure 2: Number of iterations to solve array to a precision of 0.01 vs array dimension

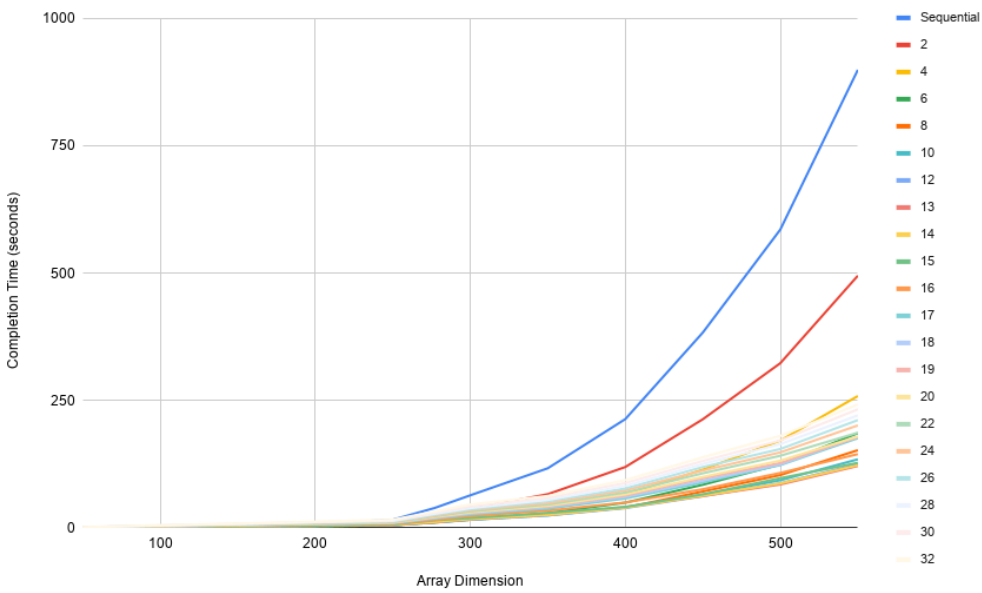

Figure 3: Program total running time vs array dimension up to a dimension of 250

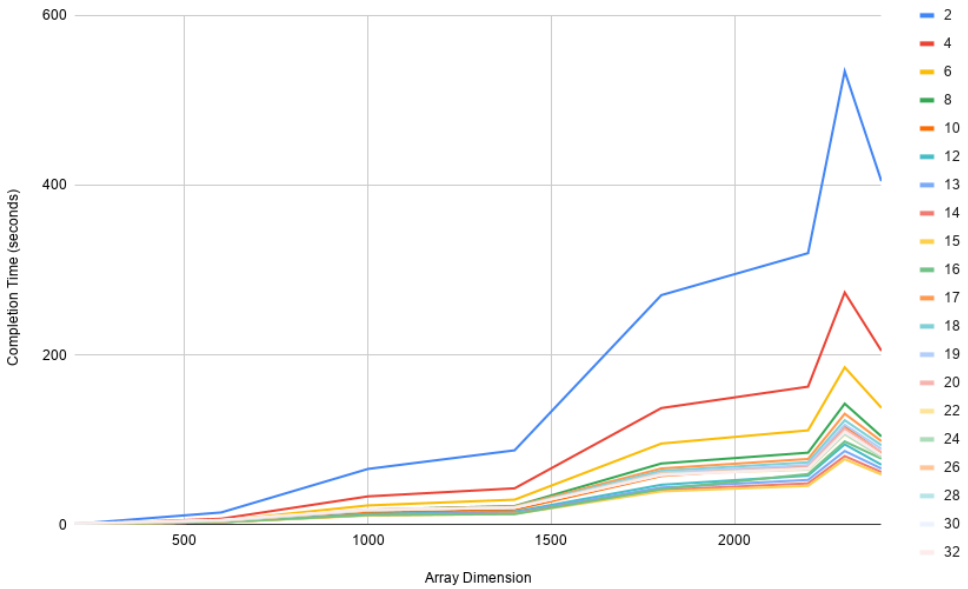

Figure 4: Program total running time vs array dimension for 1-32 threads - 1s and 0s array

Figure 5: Number of iterations to solve array to a precision of 0.01 vs array dimension - 1s and 0s array

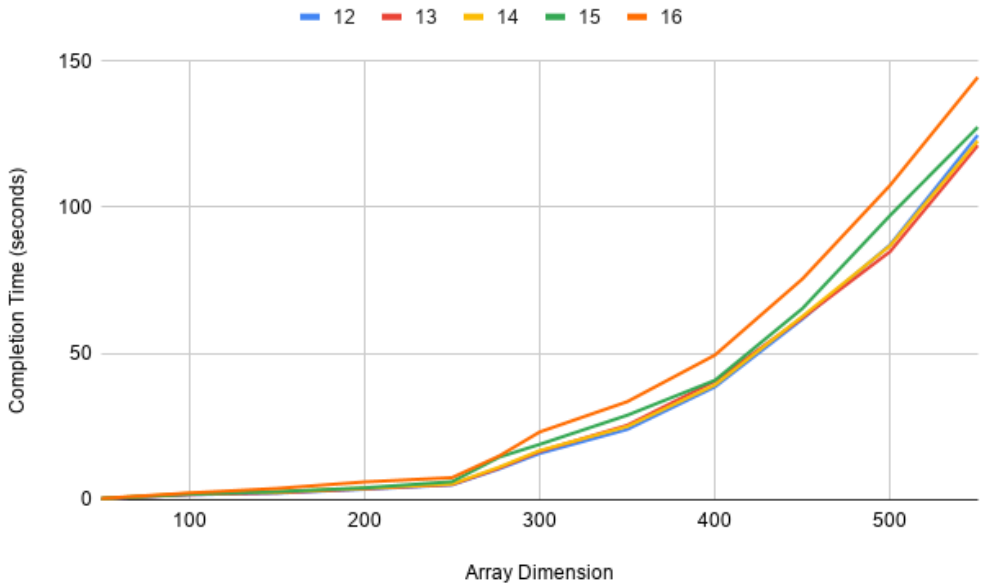

Figure 6: Program running time vs array dimension for 12-16 threads only

array sizes tested (those that could run sequentially within 15 minutes) the optimal thread count was 13, yet when tested at array sizes of 800x800 upwards (Figure 7) 15 threads became optimal. With alternate array types, 15 threads appeared as the optimal count with arrays upwards of 600x600. The increase in optimal thread count as array size scaled is even more transparent in Figure 20, as the optimal thread count for the greatest speedup shifts upwards as the array dimension increases.

It's possible that at larger array sizes, un-testable within a 15 minute time-limit with the array types developed, 16 threads would become the optimal count, as this is the number of processors present in the node. However it's also possible that 15 is in fact the highest optimal count present, with increased communication between the processor cores producing an overhead at 16 cores that only scales upwards with greater workloads. An additional explanation for a lower maximum optimal thread count may be control threads used by the Balena infrastructure occasionally preempting regular processing, resulting in the necessity of scheduling at a thread count of less than 16.

When the number of threads was scaled up beyond 16, the running time dropped again, due to the overhead of scheduling multiple threads on each processor core. The slowest average running time was for 32 threads, with

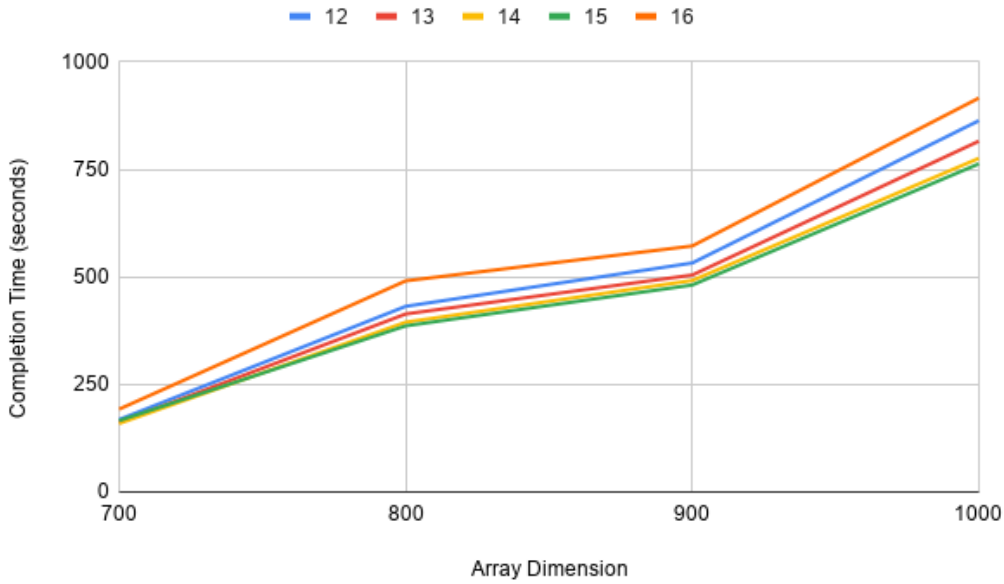

Figure 7: Program total running time vs array dimension for larger array sizes

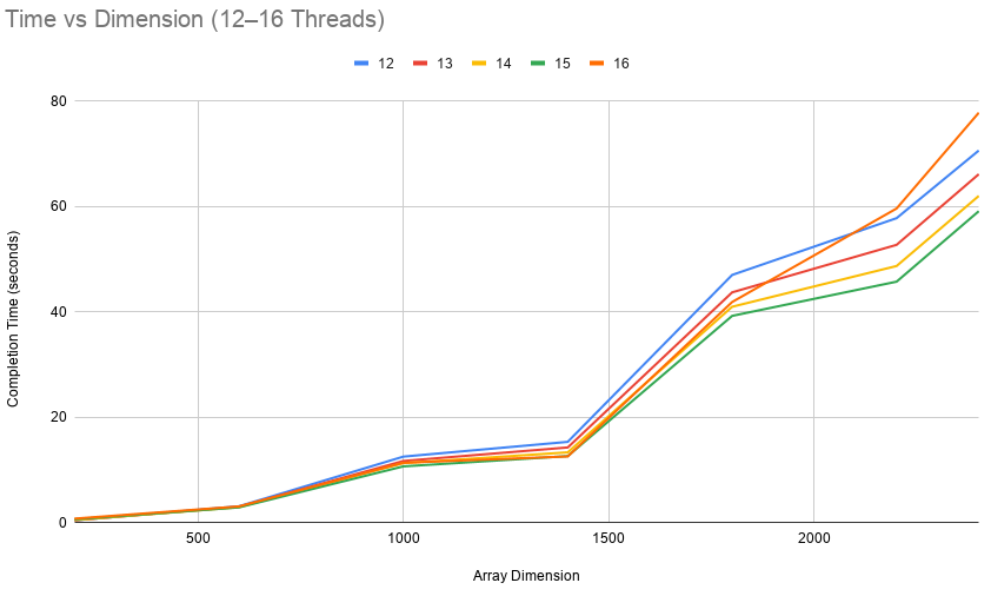

Figure 8: Program total running time vs array dimension for 12-16 threads - 1s and 0s array

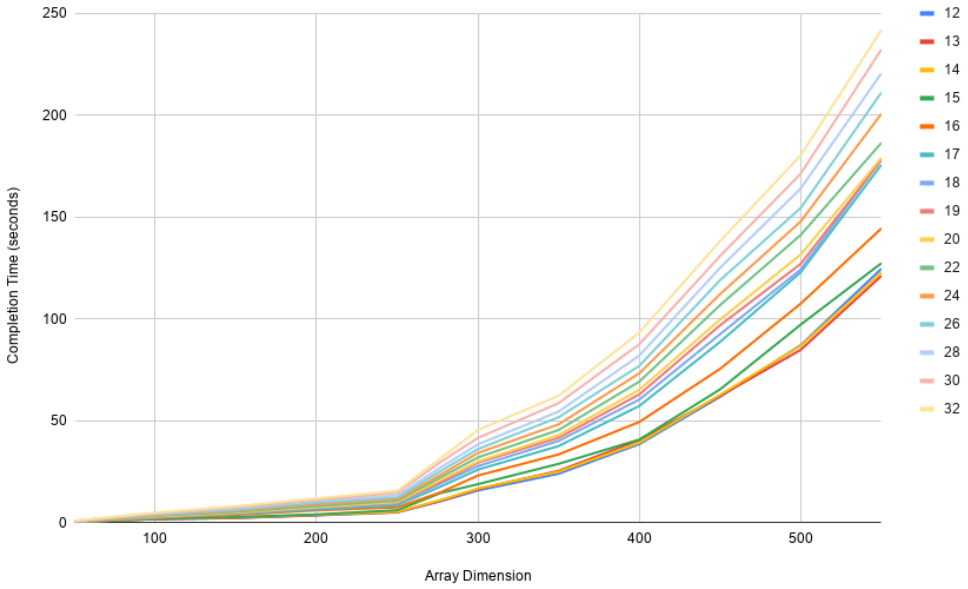

Figure 9: Program total running time vs array dimension for 12-32 Threads

the running time gradually increasing as the number of threads increased past 16 and a notable jump from 16 to 17 threads (see Figure 9). This is simply explainable by the increase in scheduling time that is required by greater thread counts. However, for alternate array types a flipped trend was displayed, with 17 threads actually running slowest of all (see Figure 10) and 32 threads running the fastest of all thread counts greater than 16, with a final running time for 32 threads only slightly slower than that for 16 threads.

One interesting, and unexpected result of the scalability investigation was the running time per *iteration*. As can be seen in both main array types tested (see Figure 11 and Figure 12) for not only 15 threads but all threads the time per iteration grew as array size increased; indeed for the 1s and 0s array type the rate of increase even grew as well. This was a surprising result, as it was to be expected that the time per iteration would stay largely stable, with perhaps a slight decrease as overheads such as thread management and scheduling became less significant as a proportion of running time. The most likely explanation for in the experiments used as iteration count grows, so does array size, meaning that the number of elements to process in each iteration grew alongside the number of total iterations the program ran. Further investigation could be conducted in the future to confirm this by exploring how arrays where total iterations dropped as array size increased behaved.

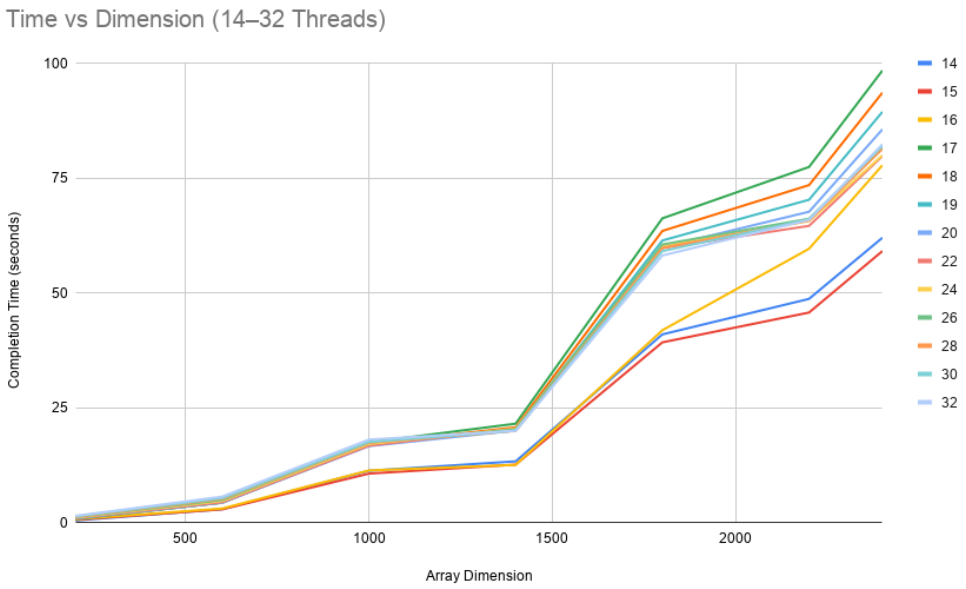

Figure 10: Program running time vs array dimension for 14-32 threads - 1s and 0s array

Another surprising result was produced by investigation into the amount of time CPUs spent idle. Using the `perf` tool, it was discovered that for a static array size, the proportion of CPU cycles spent idle increased alongside thread count (see Figure 13). In particular there was marked jump from 1 to 2 threads from 19.95% idle to 23.50% (due to the introduction of pthreads and their associated management), but the further stable rise is likely due to the average increase in the time threads spend blocked by a barrier as thread count increases. The inverse relationship is shown with increasing array size for a static thread count (see Figure 15), tending towards around 20-25% of CPU time spent idle, as a result of the relative reduction in thread management. As each thread has more elements to operate on, time spent blocked and in serial parts of the program reduces relative to the time spent processing elements in parallel.

Despite this large amount of time spent idle, average CPU utilisation (i.e. the average number of CPUs in use) was relatively high across all runs (see Figure 16 and Figure 17). Initially rising almost precisely in line with thread count, CPU utilisation was nevertheless slightly less (0.1-0.5 less) than the thread count for the 1s and 0s array, likely due to processor communication

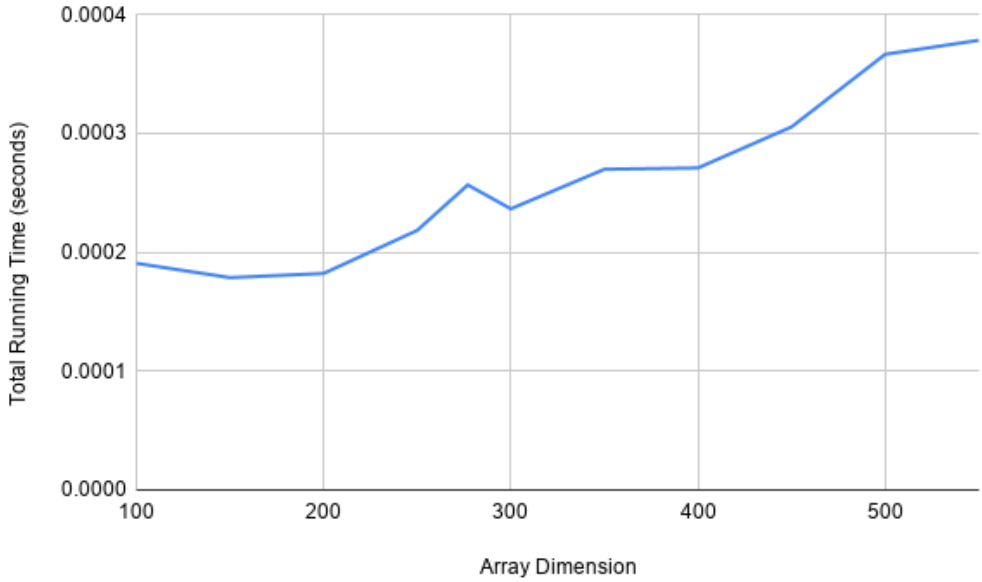

Figure 11: Time per iteration vs array dimension

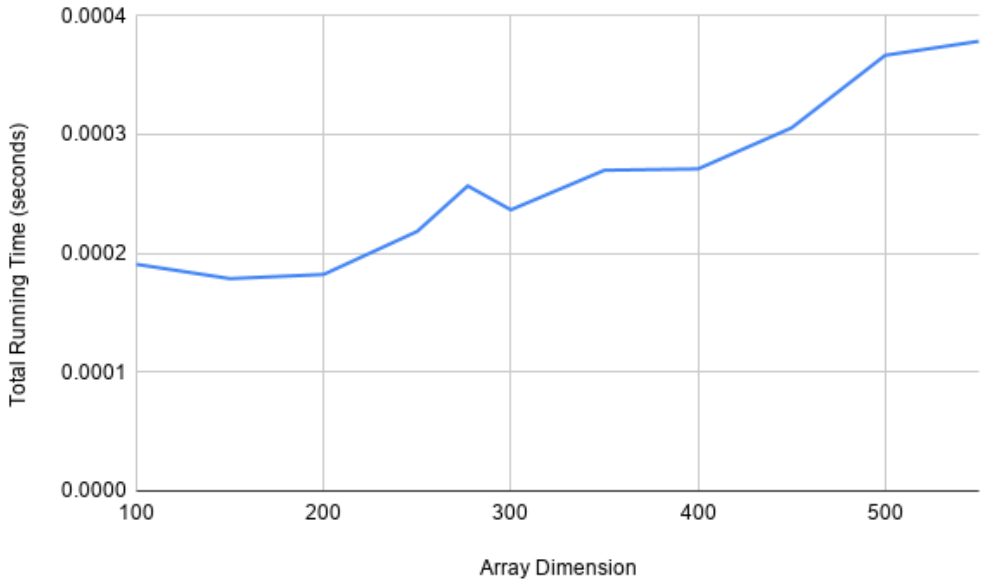

Figure 12: Time per iteration vs array dimension - 1s and 0s array

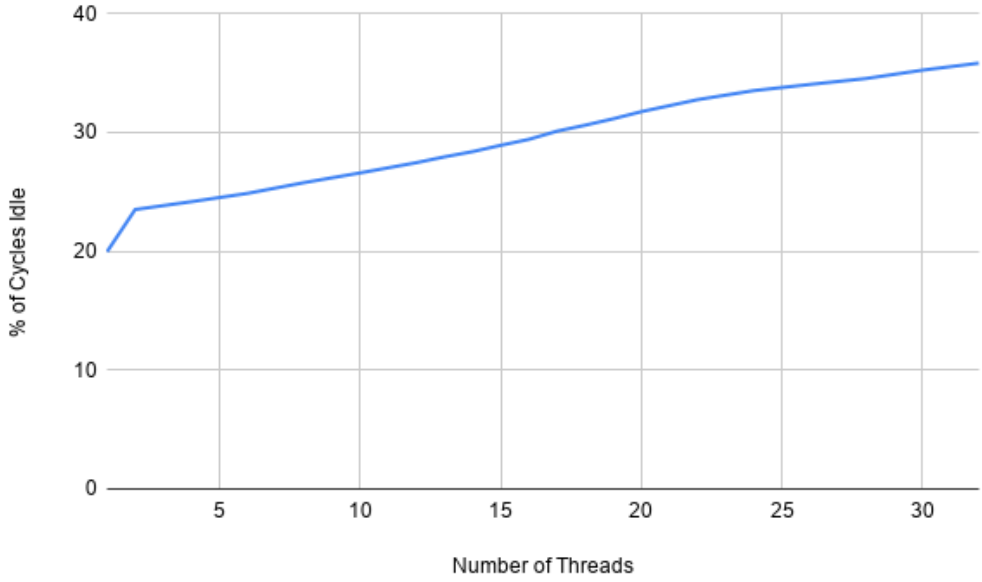

Figure 13: % of CPU cycles spent idle for constant array size (550x550)

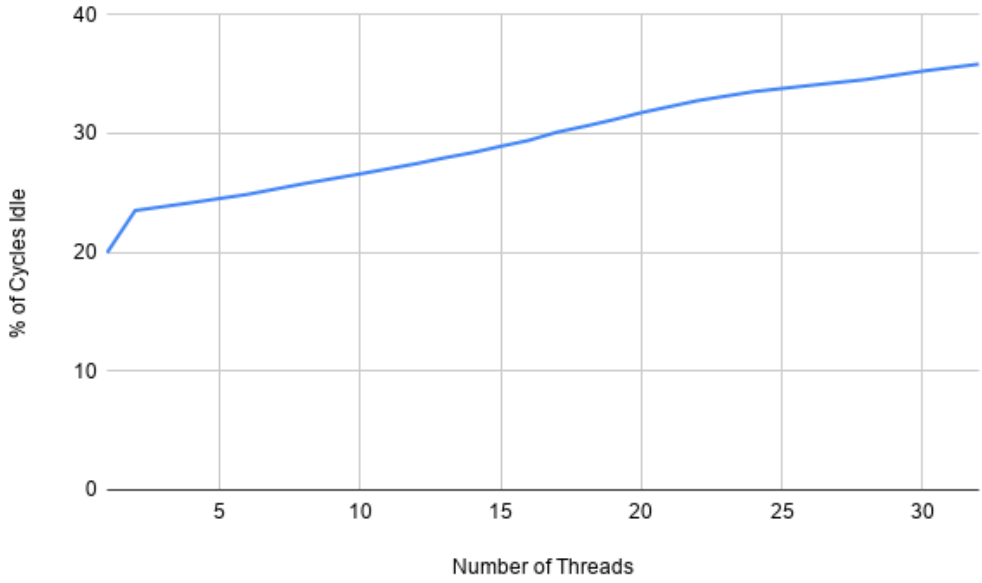

Figure 14: % of CPU cycles spent idle (2200x2200) - 1s and 0s array

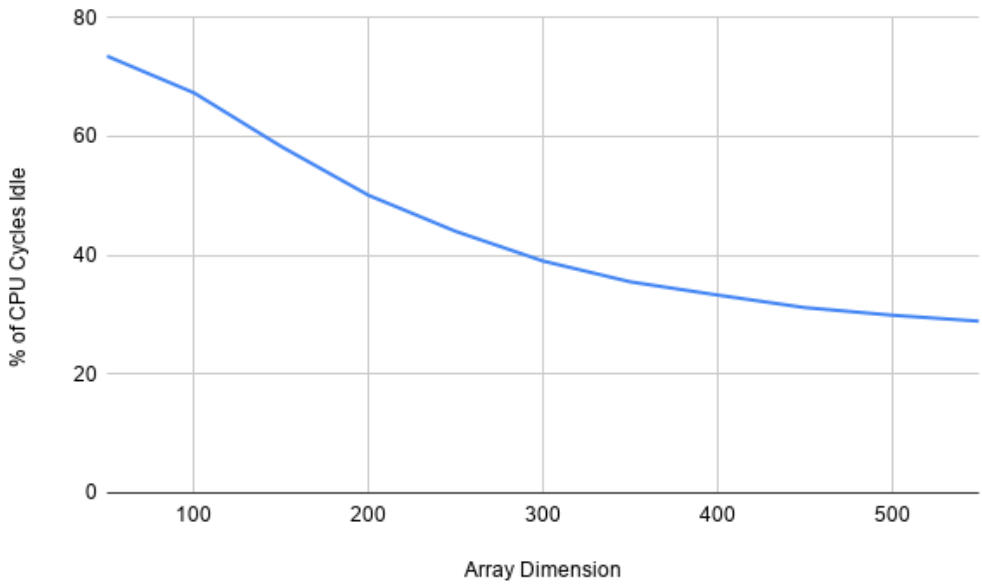

Figure 15: % of CPU cycles spent idle for constant thread count (15 threads)

overheads, and considerably less (almost half) for the default array. The highest CPU utilisation (14.219) was to be found for 15 threads for the 1s and 0s array, which further explains why this thread count displayed the best performance for this array. However, utilisation took a steep dive when more than 15 threads were in use across all array types. Again for the 1s and 0s array, average CPU utilisation was 11.057 for 16 threads, dropping to 8.491 for 17 threads and remaining relatively stable at around 10.00-10.40 from 22 threads upwards. As noted above, the effect of scheduling from 17 threads upwards likely accounts for these discrepancies.

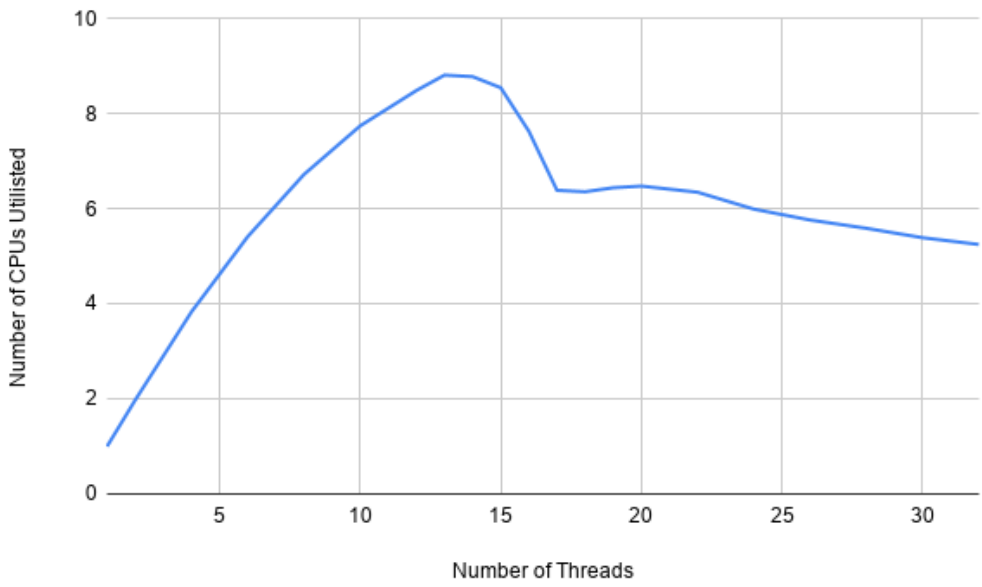

Figure 16: Average Number of CPUs Utilised (550x550)

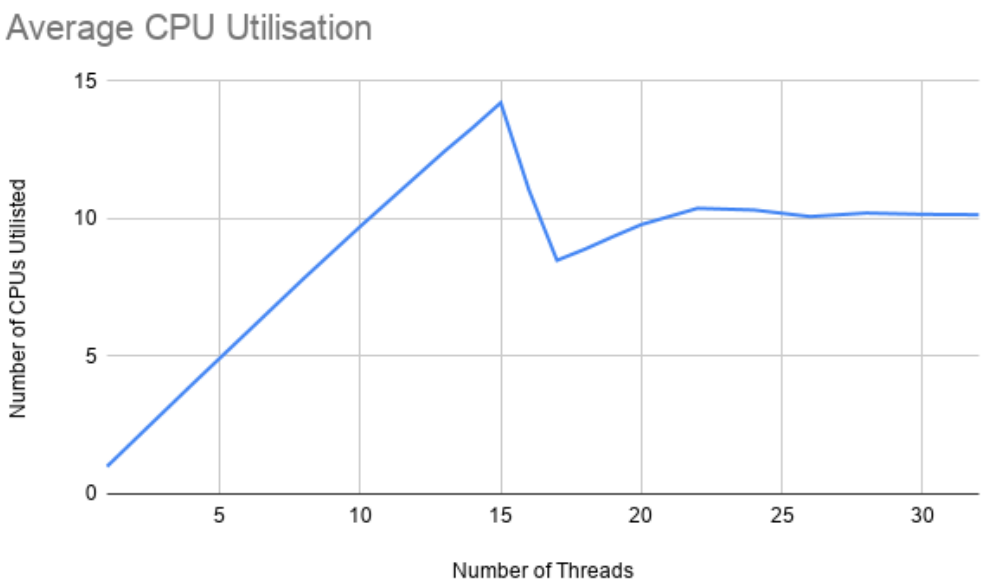

Figure 17: Average Number of CPUs Utilised (2200x2200) - 1s and 0s array

4.2 Efficiency

Efficiency was a challenge to measure for this implementation, due to the constant interaction between serial and parallel elements of the program. However, an informed estimate could be provided using a wall-clock measure within the program itself. As well as measuring the total running time of the program, the major sequential elements of the program - namely that serial processing prior to thread creation, as well as those periods when threads were blocked by wait functions - could be timed, and hence the total parallel running time be deduced by the difference between the two.

There is a markedly different trend in how efficiency varies with array size when examining the two main array types mentioned (see Figure 18 and Figure 19). One notable characteristic, however, is the thread count with peak efficiency is also the thread count with peak speedup (see Speedup section below) and the lowest running time on a given dimension. For example, with the default array 13 threads provides the lowest running time at 550x550, as does 15 threads at 2400x2400 with a 1s and 0s array - both this specifications also provide peak efficiency and peak speedup in their class. Given efficiency is a measure of speedup per processor this follows from the theoretical definition.

Efficiency on the largest array size measured for the default array (550x550) was highest for a thread count of 13, reaching a peak of 38.5%, whilst for the 1s and 0s array the peak efficiency was 77.7% with 15 threads an array size of 2200x2200.

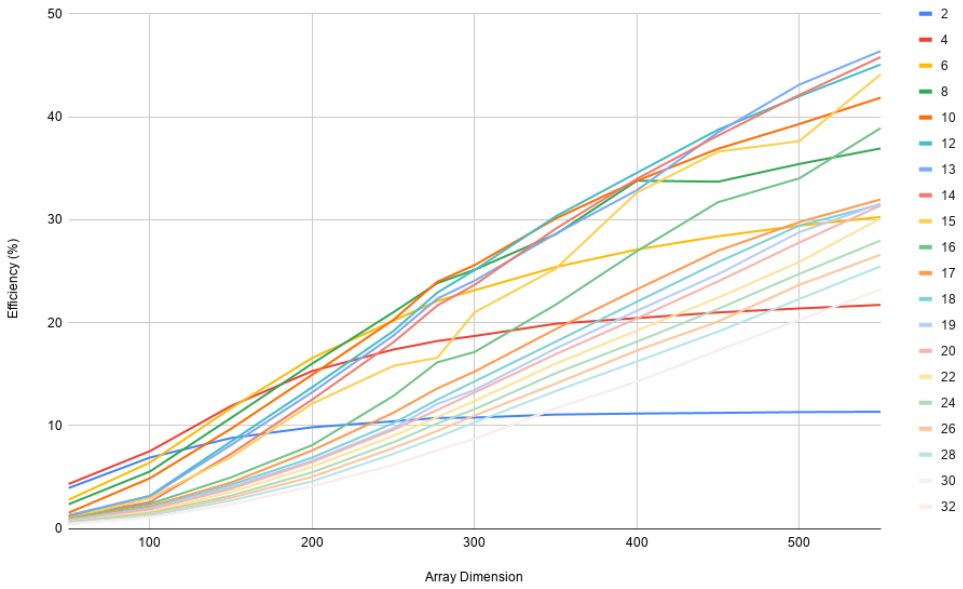

Figure 18: Efficiency vs array dimension for various thread counts

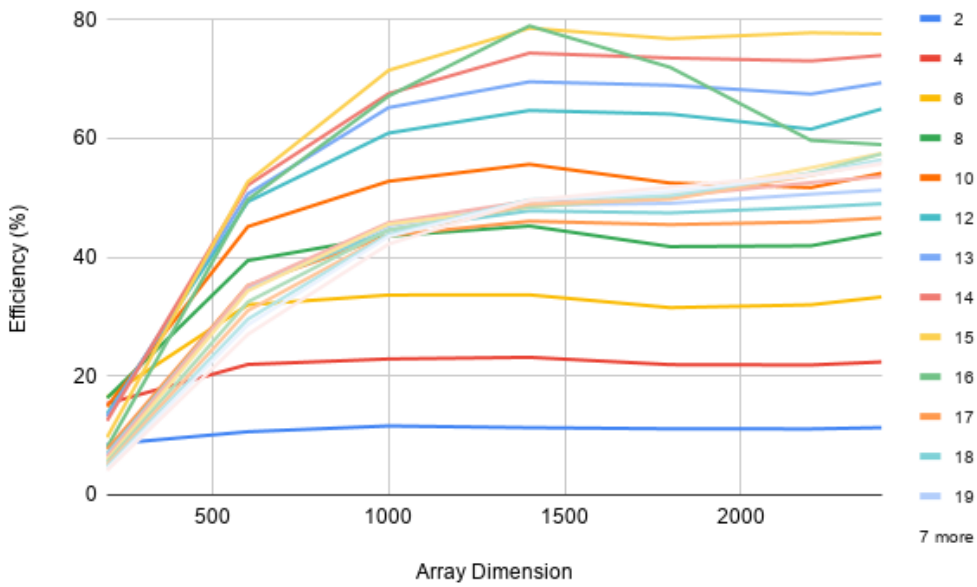

Figure 19: Efficiency vs array dimension for various thread counts - 1s and 0s array

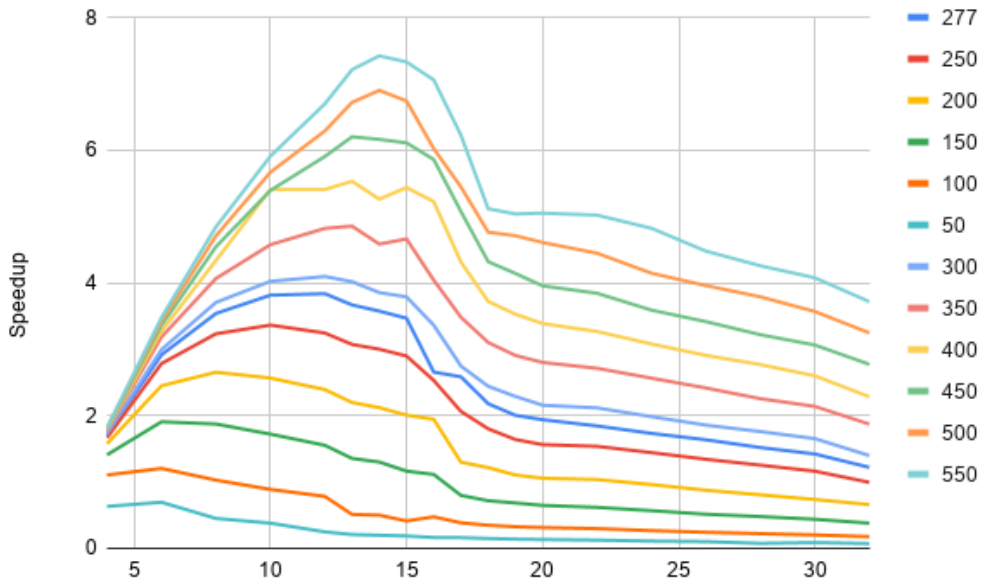

Figure 20: Speedup vs Thread Count for various array dimensions

4.3 Speedup

Speedup increases alongside thread count up to a maximum at 13 threads for the default array and 15 threads for the 1s and 0s array - the maximum speedup measured was 12.44 for an 2200x2200 array and 15 threads on the latter array type. The increase in speedup with thread count was expected, as overheads such as thread creation and serial elements of the program became relatively smaller in comparison to time spent processing in parallel. A speedup of 12.44 was also much greater than anticipated - relatively close to Amdahl's limit of 15.50 for this configuration (based off a serial running time of 0.099s and parallel running time of 45.73s), as well as an identical Gustafson's limit.

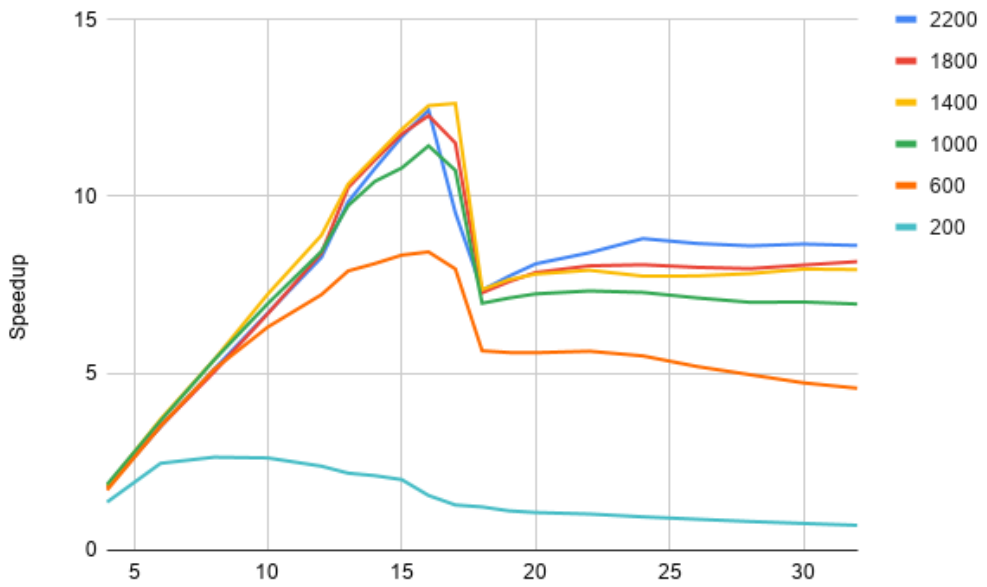

Figure 21: Speedup vs Thread Count for various array dimensions - 1s and 0s array

Data

Default Array

Time vs Dimension

	50	100	150	200	250	277	300	350	400	450	500	550
Sequential	0.069356234	0.882410452	3.063663404	7.871867181	15.55498994	38.68867732	63.58061868	116.701012	213.2424829	383.2188624	585.5236275	899.1176674
2	0.109700031	0.798133287	2.174077157	4.993291298	9.319626271	22.4779344	36.8192323	65.7890357	119.1742984	212.9542516	323.0982787	494.880201
4	0.099828195	0.732388542	1.606409998	3.213346241	5.588136953	13.26819121	21.23342829	36.64745019	65.16406163	114.0538851	171.0813265	258.563256
6	0.153761729	0.857827768	1.635146241	2.966222865	4.812246166	10.93301013	17.16040614	28.70533325	49.15972517	84.33178195	124.3557377	185.6516508
8	0.182389859	0.993037909	1.779907855	3.067956426	4.624797221	10.14363069	15.80279146	25.50857786	39.44111651	71.07656014	103.3220925	152.1935705
10	0.279306748	1.126216088	1.972812507	3.28995415	4.794457949	10.08284322	15.52388632	24.21070385	39.44111651	64.91672883	93.1218461	134.2740184
12	0.332651887	1.725438855	2.260950914	3.581831025	5.063137542	10.54972639	15.82498661	24.04098123	38.56020513	61.80332	87.15375502	124.6758935
13	0.350649972	1.763099593	2.355375331	3.712124549	5.184122735	10.82397488	16.50074522	25.45957485	40.53865525	62.20003354	84.86546084	121.1739121
14	0.370407368	2.133374249	2.632212192	3.921595604	5.367654607	11.1587239	16.78120475	25.03588543	39.22483881	62.7364199	86.87402866	122.7123525
15	0.420842671	1.857592709	2.749553347	4.048868908	6.148577359	14.56910277	18.91183956	28.88369171	40.82507686	65.42249213	97.24354279	127.4080706
16	0.428626213	2.288588119	3.83875665	6.062301268	7.539861809	14.97002075	23.15325118	33.51914614	49.45194705	75.53311181	107.5547019	144.4910075
17	0.465604302	2.531028822	4.268834397	6.47067156	8.634700722	17.75215803	26.03635021	37.62629969	57.32042234	88.73565759	122.9241906	175.7473196
18	0.498392456	2.704092141	4.480233548	7.120338111	9.487330859	19.30494498	27.77639886	40.15130246	60.45476879	92.53970006	124.3076071	178.4731881
19	0.517819875	2.807892813	4.729445335	7.448971288	9.950973864	19.96030547	29.48168927	41.66477061	62.96557177	96.90872092	127.0655775	178.0536619
20	0.557206797	2.963260312	4.960106925	7.591198157	10.11902967	20.97640063	30.03800622	43.02146919	65.26434132	99.72097527	131.6800484	179.1079979
22	0.630764753	3.302846737	5.392365835	8.183118033	10.78356482	22.3332188	32.05018645	45.51423843	69.25459564	106.7831989	141.2694289	186.4950626
24	0.681332379	3.634313716	5.956364678	8.985762595	11.58781873	23.65021227	34.2459245	48.30099187	73.35602721	112.1959803	147.8976469	200.811384
26	0.918651578	3.96520255	6.390234222	9.755270831	12.41022952	25.47069537	36.13565587	51.7596154	77.0125852	119.0666457	154.5118427	211.2449336
28	0.797459902	4.354588596	6.990351164	10.67700129	13.37488633	27.20662373	38.45449458	54.56569757	82.04471589	125.1017323	164.0846879	220.5887934
30	0.867757505	4.681314875	7.58436039	11.09057244	14.72678292	29.87243845	41.62730792	58.68836985	87.64463495	130.9284562	171.3412283	232.4004726
32	0.952085041	4.975818127	8.061584734	11.92929676	15.64197915	31.7083795	45.48776233	62.35789497	93.34823754	138.2225219	180.3874107	241.953399

Iterations vs Dimension

Dimension	50	100	150	200	250	277	300	350	400	450	500	550
Iterations	1763	9745	15397	22234	28165	56791	79988	107089	150740	214248	265416	336928

Time per Iteration (15 Threads

Dimension	50	100	150	200	250	277	300	350	400	450	500	550
Time per iteration (seconds)	0.0002387082649	0.000190620083	0.0001785772129	0.0001821025865	0.0002183056048	0.000256538937	0.0002364334595	0.0002697167002	0.0002708310791	0.0003053587064	0.0003663816152	0.0003781462822

Speedup (Threads vs Dimension)

	50	100	150	200	250	277	300	350	400	450	500	550
2	0.6322353181	1.105592846	1.409178784	1.576488675	1.669057266	1.721184724	1.726831732	1.773867192	1.789332816	1.799536096	1.812215249	1.816839036
4	0.6947559655	1.204839237	1.907149114	2.449741357	2.783573501	2.915896878	2.994364255	3.184423785	3.272393979	3.359980786	3.422487068	3.477360556
6	0.451063047	1.028656899	1.873632662	2.653835379	3.232376193	3.538703142	3.705076568	4.065481875	4.337747662	4.544180777	4.70845687	4.843036211
8	0.3802636527	0.8885969448	1.721248319	2.565834089	3.363388533	3.814085754	4.023378961	4.574971317	5.40660361	5.391634903	5.666974152	5.907724384
10	0.2483156404	0.7835178892	1.552942002	2.392698142	3.244368833	3.837080126	4.095663765	4.820223847	5.40660361	5.903237414	6.287714989	6.696140309
12	0.2084949363	0.5114121833	1.355033135	2.197721536	3.07220371	3.667268316	4.017736017	4.854253282	5.53011796	6.200619359	6.718283422	7.211640053
13	0.1977933539	0.5004881491	1.30071134	2.120582722	3.000505724	3.57435025	3.853196801	4.583776937	5.260225866	6.161071636	6.899433782	7.420059745
14	0.1872431274	0.4136219664	1.163912018	2.007312323	2.897911859	3.467123809	3.788799411	4.661349501	5.436414512	6.108395459	6.739915675	7.327034719
15	0.1648032359	0.4750290242	1.114240394	1.944213893	2.529851872	2.655529165	3.361947867	4.040377291	5.223321041	5.857601109	6.021208306	7.056991471
16	0.1618105284	0.3855697951	0.7980874234	1.298494884	2.063033824	2.5844104	2.746077352	3.48162246	4.312115006	5.073521443	5.443961233	6.222654841
17	0.1489596073	0.3486370619	0.7176814838	1.216545626	1.801450964	2.179378826	2.441994295	3.101580889	3.720183387	4.318656928	4.763290487	5.115968023
18	0.1391598793	0.3263241066	0.6838177901	1.105546824	1.639553861	2.004081201	2.289015901	2.906531168	3.527306236	4.14112929	4.710279935	5.037830484
19	0.1339389184	0.3142607324	0.6477849276	1.056772389	1.563162576	1.938280823	2.156613826	2.800951747	3.386652053	3.954431127	4.608042864	5.04970051
20	0.1244712634	0.2977836434	0.6176607582	1.036972954	1.537201733	1.844390656	2.116672399	2.712622657	3.267365894	3.842911297	4.446562974	5.019974975
22	0.1099557857	0.267166636	0.5681482855	0.961964272	1.442471965	1.732337719	1.983783114	2.564055031	3.079109494	3.588756156	4.14472991	4.821133894
24	0.1017950066	0.2427997473	0.5143512141	0.8760377428	1.34235703	1.635870194	1.856589349	2.416120405	2.906952449	3.415620251	3.958978658	4.477423786
26	0.07549786629	0.2225385566	0.4794289689	0.8069347656	1.253400666	1.518948609	1.759498123	2.254673091	2.76893033	3.218524047	3.78950647	4.256280385
28	0.08697143747	0.2026392236	0.4382703146	0.7372732257	1.162999786	1.42203155	1.653398891	2.138724826	2.599100754	3.063257841	3.568423325	4.07598978
30	0.07284667967	0.177339772	0.3800324012	0.6598768848	0.9944387337	1.220140478	1.397752174	1.871471321	2.284376101	2.77247772	3.245922901	3.716077853
32	0.07284667967	0.177339772	0.3800324012	0.6598768848	0.9944387337	1.220140478	1.397752174	1.871471321	2.284376101	2.77247772	3.245922901	3.716077853

Average CPU Utilisation (550x550)

Number of Threads	1	2	4	6	8	10	12	13	14	15	16	17	18	19	20	22	24	26	28	30	32
Average Number of CPUs Utilised	1	1.988	3.957	5.895	7.838	9.718	11.542	12.453	13.31	14.219	11.057	8.491	8.9	9.357	9.791	10.382	10.321	10.082	10.21	10.163	10.15

% of Cycles Idle (550x550)

Number of Threads	1	2	4	6	8	10	12	13	14	15	16	17	18	19	20	22	24	26	28	30	32
% of Cycles Idle	19.95	23.5	24.16	24.86	25.76	26.58	27.44	27.93	28.37	28.9	29.4	30.09	30.59	31.13	31.73	32.74	33.5	34.02	34.52	35.22	35.81

Time vs Dimension (Large Array Sizes)

	700	800	900	1000
2	771.6912036			
4	396.822793			
6	276.0181337	736.9432832		
8	219.8674193	577.828398	729.3329462	
10	187.602388	486.4276449	608.7668552	
12	168.4212374	431.7118774	532.5259574	863.4313016
13	161.5766599	413.9544536	504.4925891	815.7351379
14	159.2310312	394.8652956	491.8356571	775.8657834
15	165.981635	386.3280314	481.3083525	763.0006839
16	192.612328	490.9691786	572.063005	915.9530605
17	235.6835083	604.1354065	749.6726317	
18	233.3316872	594.3064046	731.7845494	
19	232.1181911	585.8141528	722.0401492	
20	229.8830301	579.4134055	712.6866294	
22	234.6141131	588.2875117	719.4322348	
24	241.127746	606.6233257	733.2953548	
26	248.4148253	618.4182985	743.3131791	
28	257.076369	614.0831678	751.2735058	
30	263.8890186	622.2451524	757.9050656	
32	274.0707575	640.9855649	738.5152952	

% of Cycles Idle (15 Threads)

Dimension	50	100	150	200	250	300	350	400	450	500	550
% of cycles idle	73.48	67.28	58.26	50.07	44	39.02	35.52	33.3	31.19	29.91	28.9

Efficiency (Number of Threads vs Dimension)

	50	100	150	200	250	277	300	350	400	450	550
2	3.951470738	6.909955285	8.807367399	9.853054217	10.43160791	10.75740453	10.79269832	11.08666995	11.1833301	11.2471006	28.9
4	4.342224784	7.53024523	11.91968196	15.31088348	17.39733438	18.22435549	18.71477659	19.90264865	20.45246237	20.99987991	
6	2.819144044	6.429105621	11.71020414	16.58647112	20.20235121	22.11689464	23.15672855	25.40926172	27.11092289	28.40112986	
8	2.376647829	5.553730905	10.757802	16.03646306	21.02117833	23.83803597	25.14611851	28.59357073	33.79127256	33.69771815	
10	1.551972753	4.896986807	9.705887512	14.95436339	20.27730521	23.98175079	25.59789853	30.12639905	33.79127256	36.89523384	
12	1.303093352	3.196326146	8.468957091	13.7357596	19.20127319	22.92042697	25.11085011	30.33908301	34.56323725	38.753871	
13	1.236208462	3.128050932	8.129445878	13.25364201	18.75316078	22.33968906	24.08248	28.64860586	32.87641166	38.50669773	
14	1.170269546	2.58513729	7.274450112	12.54570202	18.11194912	21.66952381	23.67999632	29.13343438	33.9775907	38.17747162	
15	1.030020225	2.968931402	6.964002461	12.15133683	15.8115742	16.59705728	21.01217417	25.25235807	32.6457565	36.61000693	
16	1.011315802	2.409811219	4.988046396	8.115593024	12.8939614	16.152565	17.16298345	21.76014037	26.95071879	31.70950902	
17	0.9309975459	2.178981637	4.485509274	7.60341016	11.25906853	13.62111766	15.26246435	19.38488055	23.25114617	26.9916058	
18	0.8697492454	2.039525666	4.273861188	6.909667647	10.24721163	12.52550751	14.30634938	18.1658198	22.04566397	25.88205806	
19	0.8371182402	1.964129578	4.048655798	6.604827429	9.769766102	12.11425514	13.47883641	17.50594842	21.16657533	24.71519454	
20	0.7779453963	1.861147771	3.860379739	6.481080966	9.60751083	11.5274416	13.22920249	16.95389161	20.42103684	24.01819561	
22	0.6872236605	1.669791475	3.550926784	6.0122767	9.015449784	10.82711075	12.39864446	16.02534395	19.24443434	22.42972598	
24	0.6362187911	1.517498421	3.214695088	5.475235892	8.389731439	10.22418871	11.60368343	15.10075253	18.1684528	21.34762657	
26	0.4718616643	1.390865979	2.996431056	5.043342285	7.833754162	9.493428809	10.99686327	14.09170682	17.30581456	20.1157753	
28	0.5435714842	1.266495147	2.739189466	4.607957661	7.268748665	8.887697187	10.33374307	13.36703016	16.24437971	19.14536151	
30	0.4552917479	1.108373575	2.375202508	4.12423053	6.215242086	7.625877987	8.735951087	11.69669575	14.27735063	17.32798575	
32	0.4552917479	1.108373575	2.375202508	4.12423053	6.215242086	7.625877987	8.735951087	11.69669575	14.27735063	17.32798575	

1s and 0s Array

Time vs Dimension

	200	600	1000	1400	1800	2200	2300	2400
Sequential	0.983106563	24.49036691	122.0585655	158.7015628	481.8576023	568.8292169	N/A	733.4716944
2	0.721059558	14.38529283	65.66870379	87.61388462	270.4644597	319.930445	534.2514938	404.8102556
4	0.399792469	6.971760358	33.34073461	42.83302096	137.372306	162.6085837	273.5230499	204.8525885
6	0.373448363	4.784908306	22.6845595	29.46441739	95.57720492	111.1593117	185.3772529	137.672898
8	0.37650942	3.882178524	17.51391005	21.92697026	72.09342821	84.82981293	142.5425674	104.0266097
10	0.413329739	3.391309952	14.45849319	17.83626918	57.40570693	68.75825409	115.9568074	84.72764459
12	0.450803488	3.102923132	12.53340408	15.33822925	47.01030679	57.78378945	94.66997539	70.62056213
13	0.465938042	3.022855524	11.70919037	14.27383212	43.70503039	52.72022352	86.79665724	66.13089453
14	0.491864326	2.936629493	11.30170429	13.34616935	40.97320854	48.70538232	80.81834601	62.00691085
15	0.633862632	2.90521962	10.68197571	12.63347976	39.24181304	45.73304734	77.1330155	59.10807676
16	0.766530272	3.080736439	11.37103318	12.57166204	41.87379196	59.61633395	98.04523568	77.79171973
17	0.801108201	4.344772713	17.48635696	21.5426237	66.22607923	77.40863795	130.6976478	98.39850314
18	0.880667935	4.382795964	17.12576493	20.75808555	63.48480293	73.46181895	123.0325962	93.57941996
19	0.919408737	4.386955056	16.84819979	20.33384355	61.4013809	70.27819497	117.733221	89.40370202
20	0.956943314	4.353602583	16.65651137	20.05695437	59.95391879	67.66542794	113.2212426	85.5882492
22	1.037871367	4.45878601	16.75274862	20.48791013	59.72565397	64.59325174	106.4967549	79.7804496
24	1.117267537	4.71363753	17.11293837	20.47376546	60.27186564	65.61721031	107.0409251	79.93359554
26	1.205654725	4.935938222	17.41955285	20.30074084	60.54967789	66.13519425	109.4329382	81.60691577
28	1.292307301	5.176629933	17.39862206	19.9744287	59.78751141	65.74599356	108.1319918	81.33446669
30	1.388638759	5.34749475	17.54191241	20.0113392	59.08285571	66.05140068	108.9495239	81.9631571
32	1.479593952	5.664891622	18.07566105	20.00283943	58.14943427	65.89722097	109.3476699	82.33639041

Iterations vs Dimension

Dimension	200	600	1000	1400	1800	2200	2300	2400
Iterations	2762	7393	13396	8912	16773	13321	20445	14356

Time per Iteration (15 Threads)

Dimension	200	600	1000	1400	1800	2200	2300	2400
Time per iteration (seconds)	0.0002294940739	0.0003929689734	0.0007974003961	0.001417580763	0.002339582247	0.003433154218	0.003772708021	0.004117308217

Average CPU Utilisation (2200x2200)

Number of Threads	1	2	4	6	8	10	12	13	14	15	16	17	18	19	20	22	24	26	28	30	32
Average Number of CPUs Utilised	1	1.973	3.827	5.409	6.716	7.738	8.482	8.806	8.774	8.541	7.626	6.384	6.354	6.438	6.473	6.344	5.989	5.76	5.586	5.388	5.246

% of Cycles Idle (2200x2200)

Number of Threads	1	2	4	6	8	10	12	13	14	15	16	17	18	19	20	22	24	26	28	30	32
Average Number of CPUs Utilised	1	1.988	3.957	5.895	7.838	9.718	11.542	12.453	13.31	14.219	1.057	8.491	8.9	9.357	9.791	10.382	10.321	10.082	10.21	10.163	10.15

Speedup (Threads vs Dimension)

	200	600	1000	1400	1800	2200	2400
2	1.363419363	1.702458699	1.85870222	1.811374573	1.781593053	1.777977763	1.8118901
4	2.459042226	3.512795284	3.660944096	3.705121872	3.507676447	3.498149999	3.580485362
6	2.632510035	5.118252084	5.380689253	5.386210788	5.041553608	5.117243067	5.327640407
8	2.61110748	6.308408219	6.969235607	7.237733299	6.683793714	6.705534261	7.050808408
10	2.378504304	7.221506514	8.441997652	8.897688255	8.393897194	8.27288628	8.656816768
12	2.180787392	7.892675992	9.738660362	10.3467982	10.25004164	9.844096803	10.38609255
13	2.109951269	8.101732522	10.42416783	11.11835711	11.02522062	10.7895828	11.09121084
14	1.998735243	8.339617567	10.80001409	11.89116956	11.76030922	11.67898063	11.82887011
15	1.550977315	8.429781604	11.42659082	12.56198339	12.27918806	12.43803442	12.40899272
16	1.28254108	7.949517069	10.73416668	12.62375351	11.50737919	9.541499439	9.428660235
17	1.227183247	5.636742938	6.980216967	7.366863249	7.275949413	7.348394598	7.454094026
18	1.116319244	5.587840983	7.127189123	7.645288985	7.590125196	7.743195378	7.837959401
19	1.069281293	5.58254338	7.244605778	7.80479905	7.84766719	8.093964525	8.204041643
20	1.027340438	5.625310634	7.327979	7.912545436	8.037132719	8.406496999	8.569770982
22	0.9472335342	5.492608718	7.285882947	7.746107916	8.067849747	8.806325763	9.193626986
24	0.8799204581	5.195640681	7.132531123	7.751459453	7.994735142	8.668902781	9.176012782
26	0.8154130222	4.961643725	7.006986148	7.81752568	7.958053933	8.601006217	8.987861966
28	0.7607374517	4.730947977	7.015415653	7.945236642	8.059502577	8.651922135	9.01796894
30	0.7079642251	4.579783254	6.958110535	7.930581817	8.155624783	8.611917554	8.948797488
32	0.6644434858	4.323183662	6.752647396	7.933951745	8.286539815	8.632066855	8.908232323

Efficiency (Number of Threads vs Dimension)

	200	600	1000	1400	1800	2200	2400
2	8.521371017	10.64036687	11.61688887	11.32109108	11.13495658	11.11236102	11.32431312
4	15.36901391	21.95497053	22.8809006	23.1570117	21.92297779	21.8634375	22.37803351
6	16.45318772	31.98907552	33.62930783	33.66381743	31.50971005	31.98276917	33.29775254
8	16.31942175	39.42755137	43.55772254	45.23583312	41.77371071	41.90958913	44.06755255
10	14.8656519	45.13441571	52.76248532	55.6105516	52.46185746	51.70553925	54.1051048
12	13.6299212	49.32922495	60.86662726	64.66748874	64.06276027	61.52560502	64.91307846
13	13.18719543	50.63582826	65.15104891	69.48973194	68.90762888	67.43489251	69.32006776
14	12.49209527	52.12260979	67.50008804	74.31980977	73.50193265	72.99362896	73.93043819
15	9.693608218	52.68613502	71.41619261	78.51239616	76.7449254	77.73771512	77.55620452
16	8.015881751	49.68448168	67.08854175	78.89845947	71.92111995	59.63437149	58.92912647
17	7.669895291	35.22964336	43.62635604	46.04289531	45.47468383	45.92746624	46.58808766
18	6.976995272	34.92400614	44.54493202	47.78305616	47.43828247	48.39497111	48.98724625
19	6.683008081	34.89089613	45.27878611	48.77999406	49.04791993	50.58727828	51.27526027
20	6.420877735	35.15819146	45.79986875	49.45340898	50.23207949	52.54060625	53.56106864
22	5.920209589	34.32880448	45.53676842	48.41317447	50.42406092	55.03953602	57.46016866
24	5.499502863	32.47275426	44.57831952	48.44662158	49.96709464	54.18064238	57.35007989
26	5.096331389	31.01027328	43.79366342	48.8595355	49.73783708	53.75628886	56.17413729
28	4.754609073	29.56842485	43.84634783	49.65772901	50.37189111	54.07451334	56.36230588
30	4.424776407	28.62364534	43.48819084	49.56613636	50.97265489	53.82448471	55.9299843
32	4.152771786	27.01989789	42.20404622	49.58719841	51.79087384	53.95041784	55.67645202